

Segmentation-based Handwritten Devanagari Optical Character Recognition.

Jeet Tandel, Aryan Sawant, Devang Vala, Utkarsh Pandey, Tanmoy Hazra, Rahul Dixit

Department of Artificial Intelligence

Sardar Vallabhbhai National Institute of Technology, Surat

Email: {u23ai042}@coed.svnit.ac.in

{tanmoyhazra, rahuldixit}@aid.svnit.ac.in

Abstract—We introduce a segmentation-driven Optical Character Recognition (OCR) framework specifically tailored for handwritten Devanagari sentence recognition. The proposed system modularizes the complex recognition process into a series of discrete stages. These stages include initial word segmentation at the sentence level, followed by adaptive detection and removal of the *shirorekha* (the headline). Subsequently, characters and *matras* (vowel signs) are extracted from dynamic spatial zones. The extracted units are then passed to two separate Convolutional Neural Network (CNN) models for prediction: one trained at the character level for consonants and vowels on 32×32 binary inputs, and another at the *matra*-level on 64×64 inputs to recognize variations above the *shirorekha*. The final recognition is achieved by an alignment algorithm that combines the prediction strengths of the character and *matra* models in a spatially-aware manner to generate complete words and sentences. The character CNN model, trained on 82,616 examples across 42 classes, achieved a test accuracy of 98.57%. The *matra* CNN model, trained on 2,999 samples distributed across four upper-*matra* classes, attained a test accuracy of 98.23%. When trained exclusively on the Devanagari Handwritten Character Dataset (DHCD), our character model achieved an accuracy of 98.80% across 46 classes, surpassing the benchmark accuracy of 98.47% reported by Acharya and Gyawali in their original work. This modular, segmentation-based pipeline demonstrates high accuracy, enhanced robustness for varying handwriting styles, and excellent scalability for sentence-level handwritten Devanagari OCR tasks.

Index Terms—Optical Character Recognition (OCR), Handwritten Character Recognition (HCR), Marathi, Devanagari, Sentence Recognition, Convolutional Neural Network (CNN), Image Segmentation, Shirorekha, Matras, Barakhadi

I. INTRODUCTION

Optical Character Recognition (OCR) has evolved into a mature research area for English scripts, yet it continues to pose significant challenges for Indic scripts such as Devanagari, which is used in languages including Hindi, Marathi, and Sanskrit. The structural complexity of Devanagari—characterized by compound consonants, numerous diacritical marks (*matras*), and the presence of a horizontal line known as the *shirorekha* that connects characters within a word—makes traditional

OCR methods less effective. These script-specific features lead to overlapping zones, variable character heights, and frequent character fusion, all of which complicate segmentation and recognition.

While deep learning-based OCR systems have achieved remarkable success in end-to-end recognition of English scripts, applying such models directly to handwritten Devanagari text often yields suboptimal results. This is primarily due to high intra-class variation in handwriting styles and the complex topology of the script, where even minor distortions in *shirorekha* can affect segmentation accuracy. Moreover, most existing models are trained at the character or isolated-word level, making them unsuitable for complete handwritten sentence recognition.

In conclusion, our key contributions can be listed as follows: (i) we present an exhaustive end-to-end handwritten Devanagari OCR system which can translate raw sentence-level images into meaningful texts, (ii) we present a script aware preprocessing and segregation approach that utilizes the structural aspects of the Devanagari writing system, such as Shirorekha-based word segregation and *matra* zone segregation, (iii) we present a dual-CNN model which independently identifies base characters and upper zone *matras*, thus enabling more accurate treatment of ambiguous elements, (iv) we prepare task-oriented datasets through the processing of Barakhadi-based handwritten data and preparation of an upper *matra* dataset since none exist in the public domain, and finally, (v) we present a rule-based reconstruction approach that incorporates geometric alignment followed by Unicode normalization to obtain meaningful sentence-level translations.

II. RELATED WORK

Handwritten Devanagari OCR is challenging due to the presence of the *shirorekha*, dependent vowel signs (*matras*), and compound characters. Existing surveys highlight that research efforts primarily focus on two core tasks: character recognition and text segmentation [1]. Several studies have examined *shirorekha*-free handwriting, underscoring the difficulty of consistent segmentation in unconstrained inputs [2].

Recent advances in handwritten Devanagari recognition predominantly rely on deep learning, particularly

convolutional neural networks (CNNs). Architectures such as DevNet, SVM and other CNN-based models have demonstrated strong performance for isolated character recognition [2]–[4]. Alternative deep architectures, including Capsule Networks, have also been explored to capture spatial relationships within characters and words [5], [6]. However, most of these methods assume pre-segmented inputs and do not address sentence-level recognition.

Segmentation of handwritten Devanagari text remains a non-trivial problem. Prior work has addressed compound and touching character segmentation using morphological, contour-based, and connected-component methods [7], [8]. Projection profiles, thresholding techniques, and statistical analysis have been widely applied for line and word segmentation [?]. While these approaches are effective in isolation, integrated segmentation-recognition pipelines are relatively limited.

Recent hybrid approaches combining segmentation with deep learning-based recognition have shown promising results [9]. In this context, our work presents a segmentation-driven end-to-end OCR framework that jointly handles shirokekha processing, matra extraction, and dual CNN-based classification for full handwritten Devanagari sentence recognition. Complementary efforts such as GAN-based synthetic data generation further emphasize the importance of robust OCR pipelines under limited data settings [10].

III. METHODOLOGY

This work proposes a segmentation-based approach for end-to-end recognition of handwritten Devanagari sentences. The system processes a scanned sentence image and reconstructs the final textual output through multiple sequential stages—preprocessing, safe cropping, word segmentation, shirokekha removal, matra and character extraction, and classification using two independent convolutional models. This is properly shown in model Flowchart Figure 1 and the image processing, segmentation and extraction pipeline Figure 8.

A. Overview of the Pipeline

The system is motivated by a segment-based and well-defined framework for handwritten Devanagari sentence recognition. The first step is image preprocessing/enhancement, which increases the visibility and contrast of strokes on the page, followed by a safe cropping to remove non-text regions outside of text area, yet keeping upper/lower modifiers. The segmented words are extracted from the preprocessed sentence image based on adaptive binary thresholding and morphological operations. For segmentation, individual words are Shirokekha (headline) detected and eliminated.

Of these, matras are taken from the area above the Shirokekha and character parts extracted from below this. Two CNNs are used for matra classification and consonant plus vowel identification separately. The final outputs are

spatially aligned with centroid-based mapping which maps the matras to their corresponding characters, it continues by a linguistic post-processing step that assembles the full sentence retaining linguistic structure and reading order of handwritten Devanagari text.

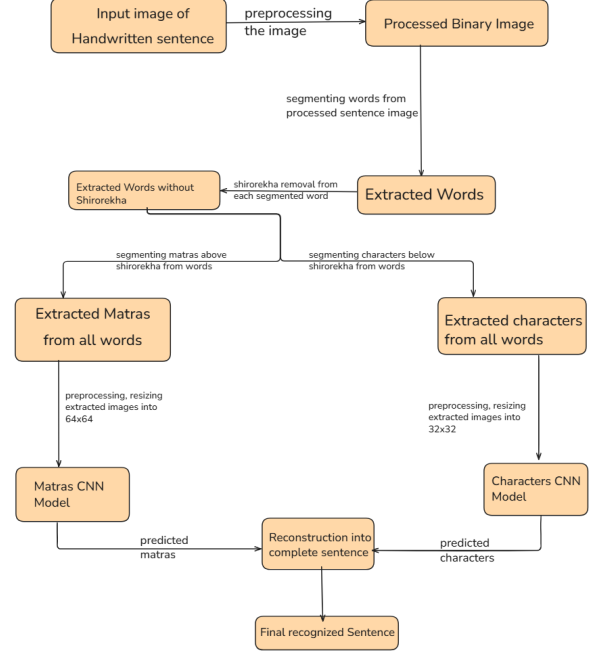


Figure 1: FlowChart of the pipeline

B. Sentence Preprocessing and Safe Cropping

Preprocessing Step: The goal of the preprocessing stage is to standardize the handwritten image as shown in Figure 2 so that it almost resembles the dataset features applied during model training. Previously researched cropping-based augmentation strategies have been shown to improve handwritten character recognition performance [11]. The raw scanned sentence’s brightness and contrast are first adjusted using a linear intensity transformation method, and a grayscale conversion is performed. Otsu’s binarization and morphological dilation are then used to highlight handwritten strokes. The shadows are saturated in fainter strokes, the characters are connected, and the quality of the separation stage is enhanced.

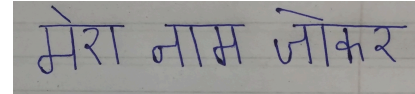


Figure 2: Raw image before preprocessing

Despite the enhancement, the image may include extraneous blank areas or uneven boundaries. Morphological closing closes small gaps between strokes and fills discontinuities to help identify the most significant text area. Contours are counted in the binary image to distinguish the region including all textual elements. Bounding rectangle cuts across the contours,

and a little padding space is inserted on all sides to ensure that the upper adders such as matras are not cut. The cropped image visualizes only the necessary handwritten content leading to a standardized and neat input for the word segmentation stage. The Final image out after this step is shown in Figure 3.



Figure 3: Illustration of sentence preprocessing and safe cropping stages. The process enhances the contrast, thickens faint strokes, and crops the image to retain only the main textual content with safe margins.

C. Word Segmentation

With the preprocessed sentence image, the words are locally recognized by first segmenting the words. First, a little blur is applied to the grayscale image to reduce noise. And with the use of adaptive thresholding method, the grayscale image is binarized to manage the uneven lighting conditions. By applying the rectangular morphological dilation, the nearby strokes are connected, ensuring the characters in a word are connected together. Then draw a boundary box around the connected word regions while also removing the smaller nested boxes. Further if the upper modifier/matras also get a separate bounding box, the matras boxes are then merged with the words bounding box. Clean, non-overlapping word segments are the result, suitable for shirokekha removal and further character extraction.

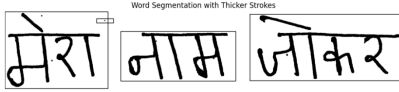


Figure 4: segmenting words from sentence

D. Shirokekha Removal

The Shirokekha, a salient horizontal line which connects consecutive character structures during writing, hinders precise segmentation and feature extraction. To address this, each word image is first binarized and a horizontal projection profile is fittingly computed over the top 70% of the image to suppress the presence of noise. A dense band of the high density and contiguous band with the highest pixel density is retained as the Shirokekha and the y-axis coordinate of this line is stored for future matras and characters segmentation. A controlled removal of this region is enforced along with a small margin above it uniformly to completely subdue the presence of residual noise or irregular strokes. Importantly, the logic ensures the subduction of only the topmost detention of the highest band, which eliminates the possibility of inadvertent removal of valid lower horizontal components of the characters. This leads to a clean image with its structure virtually preserved, allowing precise isolation of the matras and post-matra character segmentation.

$$\mathcal{S} = \left\{ r \mid \sum_{c=0}^{W-1} B(r, c) > 0.5 \max_{0 \leq r' < 0.7H} \sum_{c=0}^{W-1} B(r', c) \right\} \quad (1)$$

Here, $B(r, c) \in \{0, 1\}$ represents the binarized image, where r and c denote the row and column indices respectively, and H and W are the image height and width. The set $\mathcal{S}$ represents the detected shirokekha rows whose horizontal pixel density exceeds 50% of the maximum projection within the top 70% of the image height.



Figure 5: shirokekha removed from words with a top offset

E. Matra Extraction

After removing the Shirokekha, our next step is to isolate *matras* which mostly reside above the Shirokekha. We start by binarizing each word image using Otsu's adaptive thresholding to boost the contrast. Next, we define the region of interest above the detected Shirokekha, to account for the variations in writing styles.

Within this region, various morphological closing and dilation operations are applied to strengthen the connection of the matra components while eliminating spurious noise artifacts. After that, we proceed with finding the external contours, where we calculate the bounding boxes around the potential matra candidates in the words. For better-sophisticated results, algorithm employs adaptive filtering to filter out the components that are either too small and most likely to be not matras or completely resides within the above band of Shirokekha. Post this, we further refine the merge of closely spaced boxes, effectively capturing matras such as **ai** which might be predicted as two **ae**.

Finally, we apply small padding around all the bounding boxes to preserve the contextual boundaries to produce the clean and well-localized matra regions. This step is very essential to ensure the correct alignment of the matras with their respective base characters in the later reconstruction and recognition phases.

$$Z_m = [\max(0, y_s - \alpha h_s), y_t - \delta] \quad (2)$$

Here, Z_m denotes the adaptive matra extraction zone, where y_t and y_s represent the top and bottom coordinates of the detected Shirokekha, and $h_s = y_s - y_t$ denotes its thickness. The scaling factor α (typically 4.0–5.0) dynamically adjusts the height of the upper zone, while

δ provides a small safety margin (2–3 pixels) below the Shirorekha. This ensures coverage of the region where matras are expected to occur without encroaching on the character body.

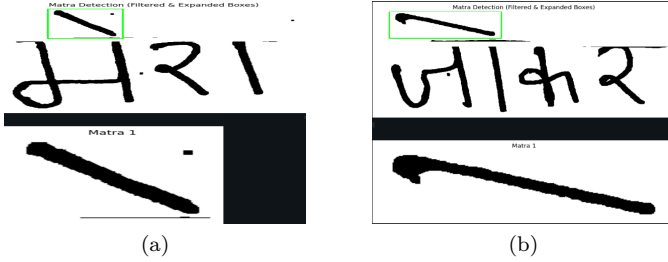


Figure 6: Extraction of matras from segmented words.

F. Character Segmentation

The character segmentation stage decouples individual characters appearing under the Shirorekha for proper separation to allow recognition. The initial step is binarizing the word image through the Otsu’s thresholding to get a binary word image. The next step is to identify the approximate position of the Shirorekha, which may have been obtained earlier in the detection method or achieved through the examination of the horizontal intensity profile of the image. Thereafter, a mask is applied below this horizontal line, whose primary role is to keep only the character region while eliminating the upper modifiers and matras.

Contours are detected within this masked area to locate distinct character regions. Finally, each segmented character is extracted and stored in left-to-right order, forming the basis for individual character recognition in later stages.

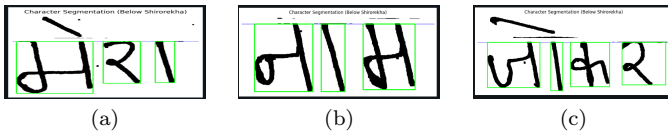


Figure 7: Extraction of characters from segmented words.

G. Model Architecture

The framework utilizes two independent CNNs to identify base characters (32×32 inputs, 42 classes) and upper-zone matras (64×64 inputs, 3 classes). Both models follow a dual-block convolutional structure (3×3 kernels, ReLU, and 2×2 max-pooling) followed by a dense layer with dropout (0.5 and 0.3) for regularization. Training was performed using the Adam optimizer with cross-entropy loss over 20 epochs. Layer configurations and parameter counts are detailed in Tables I and II.

H. Classification and Reconstruction

In the final step of this process, all the segmented components are classified and combined to yield the

Table I: Character CNN Model Summary (Input: $32 \times 32 \times 1$)

Layer (Type)	Output Shape	Param #
Conv2D (3×3, ReLU)	(30, 30, 32)	320
MaxPooling2D (2×2)	(15, 15, 32)	0
Conv2D (3×3, ReLU)	(13, 13, 64)	18,496
MaxPooling2D (2×2)	(6, 6, 64)	0
Flatten	(2304)	0
Dropout (0.5)	(2304)	0
Dense (ReLU)	(256)	590,080
Dropout (0.3)	(256)	0
Dense (Softmax)	(46)	11,822
Total Trainable Params		621,038

Table II: Matra CNN Model Summary (Input: $64 \times 64 \times 1$)

Layer (Type)	Output Shape	Param #
Conv2D (3×3, ReLU)	(62, 62, 32)	320
MaxPooling2D (2×2)	(31, 31, 32)	0
Conv2D (3×3, ReLU)	(29, 29, 64)	18,496
MaxPooling2D (2×2)	(14, 14, 64)	0
Flatten	(12544)	0
Dropout (0.5)	(12544)	0
Dense (ReLU)	(256)	3,211,520
Dropout (0.3)	(256)	0
Dense (Softmax)	(3)	771
Total Trainable Params		3,231,107

entire handwritten Devanagari sentence. All the extracted regions are preprocessed by converting them to grayscale, inverting their intensity, resizing them to their respective input dimensions, and normalizing to the $[0, 1]$ range to match the training distribution. Character segments extracted from below the Shirorekha are classified using the Character CNN. Because the ‘aa’ matra is visually ambiguous and frequently confused with a consonant, a simple geometric override based on relative size and position is used to identify and label this matra before neural inference is performed. All other segments are classified using the CNN. Upper-zone matras are independently classified by the Matra CNN.

Reconstruction takes place based on spatial and matras aware rules. Bounding boxes of the character are sorted left to right to maintain the order of reading. Matras are then linked to the base consonants using an overlap of bounding boxes horizontally, and failing that, the closest previous consonant for right matras. However, self-vowels are not taken into account for matras placement, which may lead to improper composition. Lastly, normalization of Unicode is done for resolving the composition of vowels ($\text{ॐ} + \text{ॐ} \rightarrow \text{ॐ}$ and $\text{ॐ} + \text{ॐ} \rightarrow \text{ॐ}$ are examples of such rules for correcting Devanagari character composition).

IV. DATASET

The proposed system’s modular design relies on two separate CNN models. This required the construction of two distinct datasets: a primary Base Character Dataset for consonants and vowels, and a secondary Upper Matra Dataset for diacritics found above the shirorekha.

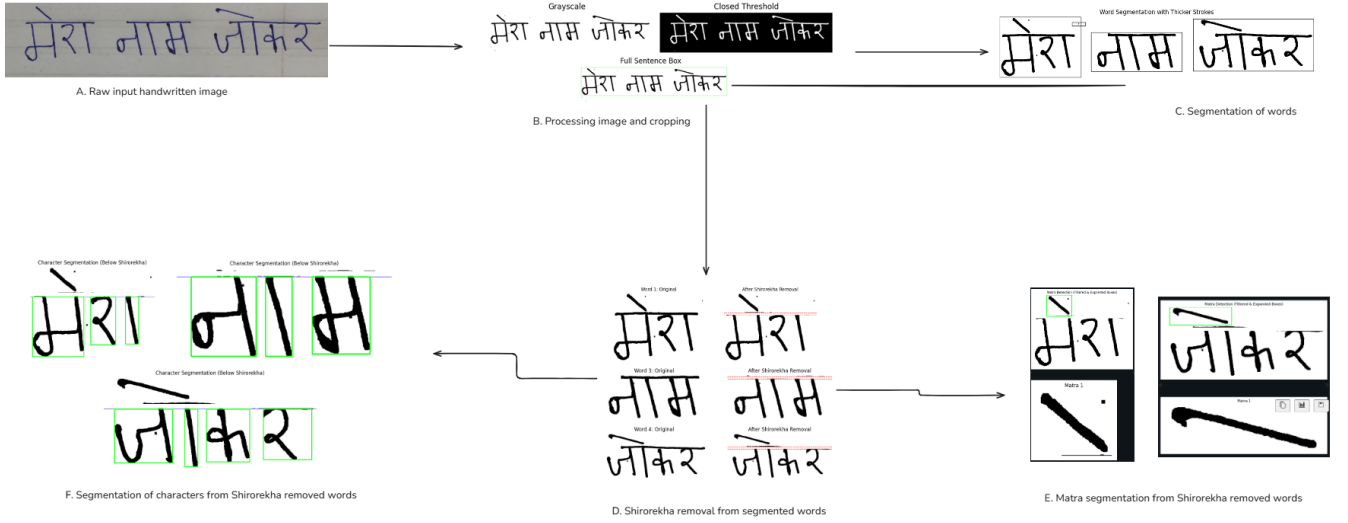


Figure 8: Complete Input Image Processing and Segmentation Flow.

A single public dataset containing all 42 target classes was not available. Therefore, a composite dataset was assembled from multiple sources.

A. Base Character Dataset

The 42-class character recognition model was trained on 36 consonants (क to ह) and five base vowels (अ, इ, उ, ऊ, ए) sourced from the UCI Machine Learning Repository [12] and a specialized academic dataset [13], respectively.

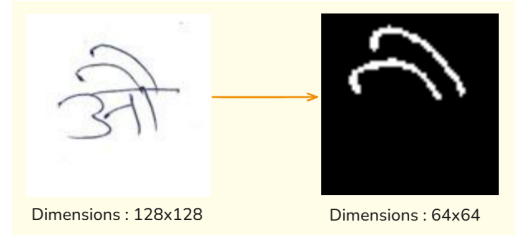


Figure 10: Matra Extraction

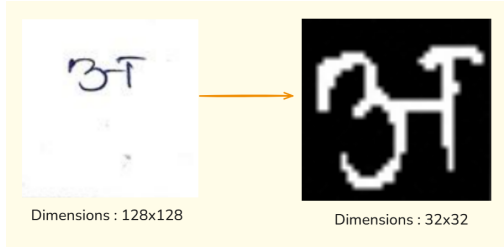


Figure 9: Barakhadi Character Image Processing

Kano Matra (ँ): This matra is structurally a vertical stroke. Samples for this class were taken from a dataset of handwritten English digits [14], using the numeral '1'.

The final Base Character Dataset contains 82,616 samples. This collection was partitioned into 66,091 training, 8,259 validation, and 8,266 test samples.

B. Upper Matra Dataset

A second, specialized dataset was created for the 3-class upper matra classification model. Standard datasets do not provide these diacritics in isolation, so they were extracted manually.

Matra Extraction: The 'ar' (रेफ), 'ae' (े), and 'ai' (ै) matras were segmented and extracted from the full-character vowel images in the academic dataset [13]. For example the extraction of matra ै is shown in Figure 10.

These matra images were processed to a 64×64 pixel resolution to preserve their details. Noisy or poorly segmented images were removed, and the clean samples were augmented.

The final Upper Matra Dataset consists of 2,999 samples across three classes. This was split into 2099 training, 449 validation, and 451 test images.

V. EXPERIMENTAL RESULTS AND ANALYSIS

The performance of the proposed segmentation-based handwritten Devanagari OCR system was evaluated separately for the two CNN classifiers, as detailed in Table III.

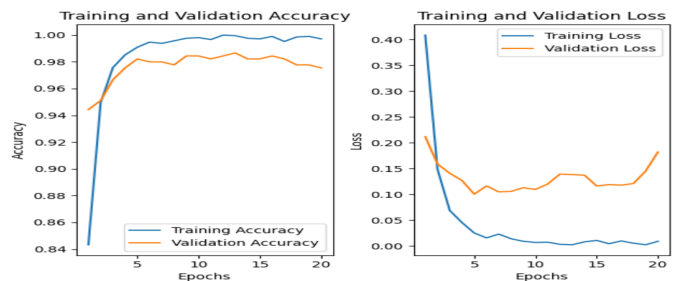


Figure 11: Training and Validation Accuracy and Loss learning curves for Matras CNN Model

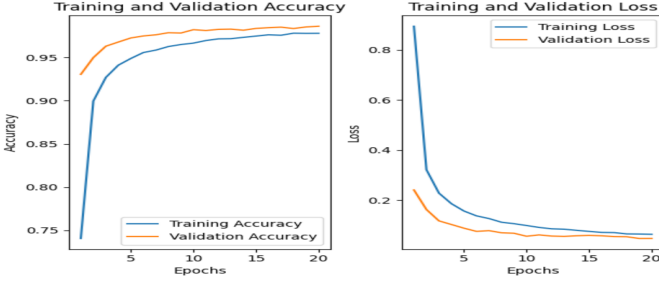


Figure 12: Training and Validation Accuracy and Loss learning curves for Characters CNN Model

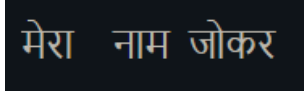


Figure 13: Final sentence after characters and matras reconstruction

A. Dataset Evaluation

This subsection presents the evaluation of the curated Barakhadi and Upper Matra datasets.

Table III: Evaluation Metrics for Upper Matra and Barakhadi Datasets

Upper Matra Dataset		Barakhadi Dataset	
Metric	Score	Metric	Score
Overall Accuracy	0.9823	Overall Accuracy	0.9711
Macro Precision	0.98	Macro Precision	0.97
Macro Recall	0.98	Macro Recall	0.97
Macro F1-score	0.98	Macro F1-score	0.97
Test Samples	451	Test Samples	866

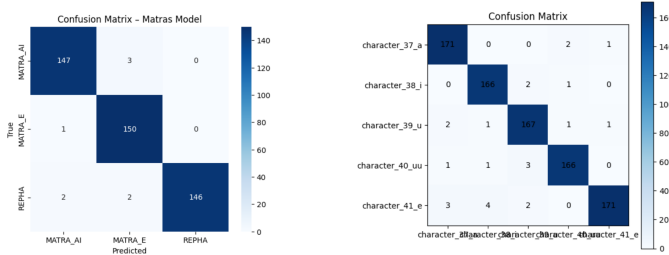


Figure 14: Confusion matrices for (left) Upper Matra classification and (right) Barakhadi classification

VI. CONCLUSION

In this paper, we present a segmentation based method for recognition of handwritten Devanagari sentences specifically designed to tackle the complexities involved with Devanagari scripts which isn't found in latin scripts. This is done by segmenting sentences to words and words to matras and characters with the help of the shirokekha as the divider. Unlike modern end-to-end OCR architectures—which learn the entire recognition process in a single model—our approach explicitly decomposes the pipeline into interpretable stages: preprocessing, word segmentation, adaptive shirokekha removal, and

separate CNN-based character and upper-zone recognition modules. Our system produced promising results, where the character CNN model reached 98.57% and matra CNN model achieved an accuracy of 98.23%. This suggests that modular, segmentation-driven recognition is effective to increasing the accuracy and robustness with respect to increasing the accuracy and robustness with respect to handwritten variation. The method is extendable to longer handwritten sentences and into other Indic scripts with trivial modification.

REFERENCES

- [1] S. Singh, N. K. Garg, and M. Kumar, "Feature extraction and classification techniques for handwritten devanagari text recognition: a survey," *Multimedia Tools Appl.*, vol. 82, no. 1, p. 747–775, Jun. 2022. [Online]. Available: <https://doi.org/10.1007/s11042-022-13318-9>
- [2] "Devanagari handwritten character recognition using convolutional neural networks," in *2020 International Conference on Electrical, Communication, and Computer Engineering (ICECCE)*, 2020.
- [3] R. Guha, N. Das, M. Kundu, M. Nasipuri, and K. C. Santosh, "Devnet: An efficient cnn architecture for handwritten devanagari character recognition," *International Journal of Pattern Recognition and Artificial Intelligence*, 2020.
- [4] M. S. Ansari, M. Wasid, and S. A. Rahman, "Devanagari handwritten character recognition using transfer learning with deep cnn and svm," in *2022 5th International Conference on Multimedia, Signal Processing and Communication Technologies (IMPACT)*, 2022, pp. 1–5.
- [5] "Handwritten devanagari manuscript characters recognition using capsnet," *Iranian Journal of Chemistry and Chemical Engineering*, 2023.
- [6] B. Mandal, S. Dubey, S. Ghosh, R. Sarkhel, and N. Das, "Handwritten indic character recognition using capsule networks," 12 2018.
- [7] M. S. Bhandare and A. S. Kakade, "Handwritten (marathi) compound character recognition," in *2015 International Conference on Innovations in Information, Embedded and Communication Systems (ICIECS)*, 2015, pp. 1–4.
- [8] S. Babu and M. Jangid, "Touching character segmentation of devanagari script," in *Proceedings of the 7th International Conference on Computing Communication and Networking Technologies*. Association for Computing Machinery, 2016.
- [9] P. Yawalkar and M. Kharat, "A hybrid approach for recognition of hand written devnagri compound characters," *Asian Journal of Computer Science and Technology*, vol. 8, pp. 70–76, 05 2019.
- [10] S. Fogel, H. Averbuch-Elor, S. Cohen, S. Mazor, and R. Litman, "Scrabblegan: Semi-supervised varying length handwritten text generation," 2020. [Online]. Available: <https://www.amazon.science/publications/scrabblegan-semi-supervised-varying-length-handwritten-text-generation>
- [11] S. Acharya, A. K. Pant, and P. K. Gyawali, "Deep learning based large scale handwritten devanagari character recognition," in *2015 9th International Conference on Software, Knowledge, Information Management and Applications (SKIMA)*, 2015, pp. 1–6.
- [12] S. Acharya and P. Gyawali, "Devanagari Handwritten Character Dataset," UCI Machine Learning Repository, 2015.
- [13] S. W. Khandakhani, S. Dash, S. Padhy, and R. Panda, "MHCD_GIETV1," Mendeley Data, V1, 2024.
- [14] Y. LeCun, C. Cortes, and C. J. Burges, "Mnist handwritten digit database," *ATT Labs*, 2010. [Online]. Available: <http://yann.lecun.com/exdb/mnist>